راهنمای راه‌اندازی نخستین پروژه ThreadX

روی میکروکنترلر STM۳۲H۵ با استفاده از STM۳۲CubeMX

مرحله صفر: از آزمون بدون سیستم‌عامل تا اجرای نخستین Thread

۱. مقدمه و هدف

این سند مراحل ایجاد و اجرای نخستین پروژه مبتنی بر سیستم‌عامل بلادرنگ ThreadX را روی یک برد با میکروکنترلر STM۳۲H۵ شرح می‌دهد. هدف نهایی این مرحله آن است که یک Thread تحت مدیریت ThreadX، LED قرمز برد را هر ۵۰۰ میلی‌ثانیه تغییر وضعیت دهد و هم‌زمان مقدار تیک سیستم را از طریق پورت سریال UART۷ روی ترمینال کامپیوتر نمایش دهد.

مسیر کار عمداً در دو مرحله طراحی شده است. در مرحله نخست، پروژه بدون هیچ سیستم‌عاملی ساخته می‌شود تا سلامت سخت‌افزار، کلاک، پورت سریال و LEDها تأیید شود. در مرحله دوم، ThreadX به پروژه افزوده می‌شود. مزیت این روش آن است که اگر در مرحله دوم مشکلی پیش آمد، با اطمینان می‌توان گفت سخت‌افزار سالم است و علت را باید در تنظیمات سیستم‌عامل جست‌وجو کرد.

۱.۱ مشخصات سخت‌افزار و نرم‌افزار

مورد	مقدار
میکروکنترلر	خانواده STM۳۲H۵ (با نام معرفی‌شده STM۳۲H۵RIT)
پورت سریال برای printf	`UART7` روی پایه‌های `PB3` و `PB4`، با نرخ `115200` و قالب `8N1`
LED قرمز	`PA8`
LED آبی	`PC6`
LED سبز	`PC9`
ابزار پیکربندی	STM۳۲CubeMX
محیط توسعه	STM۳۲CubeIDE (یا هر محیط سازگار با پروژه تولیدشده)
نسخه ThreadX	۶.۴.۰ (نسخه ارائه‌شده در CubeMX)

مورد	مقدار
اتصال برنامه‌ریز و دیباگر	ST-Link از طریق SWD
نرم‌افزار ترمینال	PuTTY، Tera Term یا ترمینال داخلی CubeIDE

نکته: در تمام مراحل این سند، قابلیت TrustZone غیرفعال در نظر گرفته شده است. فعال‌سازی آن پیچیدگی اضافی ایجاد می‌کند و برای یادگیری مفاهیم پایه ضرورتی ندارد.

۲. مرحله نخست: آزمون بدون سیستم‌عامل

۲.۱ پیکربندی در CubeMX

پیش از نوشتن کد، تنظیمات زیر در CubeMX انجام می‌شود:

- **UART۷:** حالت `Asynchronous`، نرخ ۱۱۵۲۰۰ بیت بر ثانیه، ۸ بیت داده، بدون parity و یک بیت stop. پایه‌های `PB3` و `PB4` به این پورت اختصاص داده می‌شوند. پایه TX میکروکنترلر باید به پایه RX مبدل USB-سریال متصل شود و GND دو طرف مشترک باشد.
- **LEDها:** پایه‌های `PA8`، `PC6` و `PC9` به‌صورت خروجی GPIO تنظیم می‌شوند و برای خوانایی کد نام‌های مناسب دریافت می‌کنند.
- **SYS:** گزینه `Debug` روی `Serial Wire` قرار می‌گیرد تا برنامه‌ریزی و دیباگ از طریق SWD ممکن باشد.

۲.۲ هدایت خروجی printf به UART۷

توابع کتابخانه استاندارد زبان C، از جمله `printf`، خروجی خود را در نهایت از طریق تابع سطح پایین `_write` ارسال می‌کنند. با تعریف این تابع در پروژه و فراخوانی `HAL_UART_Transmit` در درون آن، هر متنی که `printf` تولید کند روی UART۷ ارسال خواهد شد. کد زیر در فایل `uart.c` و داخل بخش `USER CODE 0` قرار می‌گیرد.

کد ۱ — تعریف تابع _write در فایل uart.c

```
/* USER CODE BEGIN 0 */
extern UART_HandleTypeDef huart7;


/* Redirect printf output to UART7 */
int _write(int fd, char *ptr, int len)
{
    (void)fd;


    if (HAL_UART_Transmit(&huart7, (uint8_t *)ptr, (uint16_t)len,
                          HAL_MAX_DELAY) != HAL_OK)
    {
        return -1;
    }


    return len;
}
/* USER CODE END 0 */
```

توضیح اجزای این کد:

- **بلوک‌های USER CODE**: CubeMX هنگام تولید مجدد کد، هر نوشته‌ای را که خارج از این بلوک‌ها باشد حذف می‌کند. به همین دلیل کد شخصی باید همواره میان `USER CODE BEGIN` و `USER CODE END` نوشته شود.
- **extern UART_HandleTypeDef huart۷**: این خط اعلام می‌کند که ساختار مدیریت UART۷ در فایل دیگری تعریف شده است. نام `huart7` را باید با نام موجود در کد تولیدشده تطبیق داد.
- **پارامتر fd**: شناسه جریان خروجی است. چون تمام خروجی‌ها به یک پورت ارسال می‌شوند، این پارامتر استفاده نمی‌شود و با `(void)fd` از هشدار کامپایلر جلوگیری می‌گردد.
- **HAL_UART_Transmit**: سه پارامتر اصلی آن به ترتیب اشاره‌گر به UART، آدرس داده و تعداد بایت‌ها هستند. پارامتر چهارم، `HAL_MAX_DELAY`، به این معناست که تابع تا پایان ارسال منتظر می‌ماند (ارسال همگام).
- **مقدار بازگشتی:** تابع `_write` باید تعداد بایت‌های ارسال‌شده را برگرداند، و در صورت خطا مقدار `-1`.

اگر هنگام ساخت پروژه خطای `multiple definition of _write` مشاهده شد، به این معناست که فایل `syscalls.c` پروژه تعریف خود را دارد. در این حالت تابع `_write` حذف می‌شود و به‌جای آن تابع زیر نوشته می‌شود:

کد ۲ — روش جایگزین با `__io_putchar`

```
/* Alternative: send one character at a time */
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart7, (uint8_t *)&ch, 1, HAL_MAX_DELAY);

    return ch;
}
```

۲.۳ کد فایل main.c

ابتدا کتابخانه `stdio.h` به فایل اضافه می‌شود تا `printf` شناخته شود:

کد ۳ — افزودن سرفایل در main.c

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */
```

سپس پس از فراخوانی `MX_UART7_Init()` و پیش از حلقه اصلی، بافر خروجی غیرفعال شده و پیام آغازین چاپ می‌شود:

کد ۴ — راه‌اندازی اولیه در بخش USER CODE ۲

```
/* USER CODE BEGIN 2 */
setvbuf(stdout, NULL, _IONBF, 0);   /* Disable output buffering */
printf("Hello, bare metal!\r\n");
/* USER CODE END 2 */
```

در نهایت داخل حلقه `while(1)` منطق چشمک‌زن و چاپ پیام قرار می‌گیرد:

کد ۵ — حلقه اصلی در بخش USER CODE ۳

```
/* USER CODE BEGIN 3 */
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8);   /* Red LED */
printf("tick %lu\r\n", HAL_GetTick());
HAL_Delay(500);
/* USER CODE END 3 */
```

توضیح اجزای این کد:

- **setvbuf با گزینه _IONBF**: بدون این دستور، کتابخانه C ممکن است خروجی را تا پر شدن بافر یا رسیدن کاراکتر خط جدید نگه دارد و کاربر گمان کند که ارسال انجام نمی‌شود.
- **دنباله \r\n**: کاراکتر `\r` مکان‌نما را به ابتدای خط برمی‌گرداند و `\n` خط جدید ایجاد می‌کند. بسیاری از ترمینال‌ها برای نمایش صحیح به هر دو نیاز دارند.
- **HAL_GPIO_TogglePin**: وضعیت پایه را معکوس می‌کند. در اینجا `PA8` که به LED قرمز متصل است هدف قرار گرفته است.
- **HAL_GetTick**: مقدار شمارنده زمان سیستم را بر حسب میلی‌ثانیه برمی‌گرداند.
- **HAL_Delay**: تأخیر از نوع انتظار فعال (busy-wait) است؛ یعنی پردازنده در این مدت کار دیگری انجام نمی‌دهد. این رفتار در مرحله بعد با تأخیر سیستم‌عامل مقایسه خواهد شد.

۲.۴ خروجی مورد انتظار

پس از برنامه‌ریزی برد و اتصال ترمینال با نرخ ۱۱۵۲۰۰ و قالب ۸N۱، خروجی مشابه نمونه زیر دیده می‌شود. فاصله حدود ۵۰۱ میلی‌ثانیه میان پیام‌ها طبیعی است، زیرا `HAL_Delay` یک تیک اضافه می‌گیرد و خود `printf` نیز حدود یک میلی‌ثانیه زمان می‌برد.

خروجی ۱ — نمونه خروجی روی ترمینال

```
Hello, bare metal!
tick 1
tick 503
tick 1004
tick 1505
```

۳. مرحله دوم: افزودن ThreadX

۳.۱ تنظیمات CubeMX

پیش از فعال‌سازی ThreadX، تنظیمات زیر باید بررسی یا اعمال شود. مهم‌ترین مورد، تغییر منبع زمان HAL از SysTick به یک تایمر دیگر است.

تنظیم	مقدار	توضیح
SYS ← Timebase Source	`TIM6` یا `TIM7`	ThreadX از SysTick برای تیک خود استفاده می‌کند. اگر HAL نیز از آن استفاده کند، تداخل رخ می‌دهد و سیستم رفتار نادرست یا متوقف خواهد داشت.
Middleware ← ThreadX	فعال (`Core`)	کرنل ThreadX به پروژه افزوده می‌شود.
Generate App Init Code	فعال	فایل `app_threadx.c` و تابع `App_ThreadX_Init` تولید می‌شود؛ بدون آن محلی برای ساخت Threadها وجود ندارد.
ThreadX memory pool size	`16384` بایت	مقدار پیش‌فرض ۱۰۲۴ بایت برای ساخت حتی یک Thread همراه با `printf` کافی نیست. این استخر حافظه Stack هر Thread را تأمین می‌کند.
TX_ENABLE_STACK_CHECKING	Enabled (توصیه‌شده)	سرریز Stack را زودتر آشکار می‌کند و در مراحل دیباگ مفید است.
TX_TIMER_TICKS_PER_SECOND	`100` (پیش‌فرض)	هر تیک سیستم برابر ۱۰ میلی‌ثانیه است. بهتر است فعلاً تغییر نکند.
سایر تنظیمات	پیش‌فرض	مقادیر دیگر در این مرحله نیاز به تغییر ندارند.

یادآوری: در GitHub نسخه جدیدتری از (۶.۵.۲) ThreadX در دسترس است، ولی CubeMX نسخه ۶.۴.۰ را ارائه می‌دهد. نسخه‌ای که ST همراه با درایورها و HAL تراشه آزموده است، مطمئن‌ترین انتخاب برای یادگیری است و رابط‌های برنامه‌نویسی مورد استفاده در این مرحله میان این دو نسخه تفاوت مهمی ندارند.

۳.۲ توالی راه‌اندازی برنامه

درک ترتیب اجرا برای نوشتن درست کد ضروری است. در پروژه تولیدشده توسط CubeMX، مسیر اجرا به شرح زیر است (نام دقیق توابع ممکن است بسته به نسخه CubeMX اندکی تفاوت داشته باشد):

- **گام نخست:** تابع `main()` سخت‌افزار (کلاک، GPIO و UART) را راه‌اندازی می‌کند.
- **گام دوم:** تابع `MX_ThreadX_Init()` فراخوانی می‌شود و کرنل با `tx_kernel_enter()` آغاز می‌گردد.
- **گام سوم:** کرنل تابع `tx_application_define()` را اجرا می‌کند که در نهایت `App_ThreadX_Init()` را فرا می‌خواند. این همان محلی است که Threadها و سایر منابع سیستم ساخته می‌شوند.
- **گام چهارم:** زمان‌بند (scheduler) فعال می‌شود و کنترل برنامه دیگر به `main` بازنمی‌گردد. از این لحظه Threadها اجرای برنامه را بر عهده دارند.

بنابراین مرز اصلی چنین است: پیش از `tx_kernel_enter` برنامه به شیوه معمول اجرا می‌شود و پس از آن در دنیای Threadها قرار داریم. حلقه `while(1)` داخل `main` پس از آغاز کرنل هرگز اجرا نمی‌شود. دستورهای بخش `USER CODE 2` تنها در صورتی اجرا می‌شوند که پیش از فراخوانی `MX_ThreadX_Init()` قرار داشته باشند؛ در غیر این صورت باید جابه‌جا شوند.

۳.۳ کد فایل app_threadx.c

تمام تغییرات این بخش در فایل `app_threadx.c` و داخل بلوک‌های `USER CODE` انجام می‌شود. ابتدا سرفایل‌ها اضافه می‌شوند:

کد ۶ — سرفایل‌ها

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "main.h"
/* USER CODE END Includes */
```

سپس ثابت‌های اندازه Stack و اولویت تعریف می‌شوند:

کد ۷ — ثابت‌ها

```
/* USER CODE BEGIN PD */
#define BLINK_STACK_SIZE    2048
#define BLINK_PRIORITY      10
/* USER CODE END PD */
```

ساختار کنترلی Thread به‌صورت متغیر ایستا (static) تعریف می‌شود تا در تمام طول اجرای برنامه معتبر بماند، و نمونه تابع Thread اعلام می‌گردد:

کد ۸ — متغیر و اعلان تابع

```
/* USER CODE BEGIN PV */
static TX_THREAD blink_thread;
/* USER CODE END PV */


/* USER CODE BEGIN PFP */
static VOID blink_thread_entry(ULONG thread_input);
/* USER CODE END PFP */
```

ساخت Thread در تابع `App_ThreadX_Init`، داخل بخش `App_ThreadX_MEM_POOL` انجام می‌شود:

کد ۹ — ساخت Thread

```
/* USER CODE BEGIN App_ThreadX_MEM_POOL */
TX_BYTE_POOL *byte_pool = (TX_BYTE_POOL *)memory_ptr;
CHAR *stack_ptr = TX_NULL;


/* Allocate the thread stack from the byte pool */
if (tx_byte_allocate(byte_pool, (VOID **)&stack_ptr,
                     BLINK_STACK_SIZE, TX_NO_WAIT) != TX_SUCCESS)
{
    return TX_POOL_ERROR;
}


/* Create the blink thread */
if (tx_thread_create(&blink_thread, "blink", blink_thread_entry, 0,
                     stack_ptr, BLINK_STACK_SIZE,
                     BLINK_PRIORITY, BLINK_PRIORITY,
                     TX_NO_TIME_SLICE, TX_AUTO_START) != TX_SUCCESS)
{
    return TX_THREAD_ERROR;
}
/* USER CODE END App_ThreadX_MEM_POOL */
```

و سرانجام تابعی که کار Thread را تعریف می‌کند، در انتهای فایل و داخل بخش `USER CODE 1` نوشته می‌شود:

کد ۱۰ — تابع ورودی Thread

```
/* USER CODE BEGIN 1 */
static VOID blink_thread_entry(ULONG thread_input)
{
    (void)thread_input;


    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8);   /* Red LED */
        printf("tick %lu\r\n", (unsigned long)tx_time_get());
        tx_thread_sleep(50);                     /* 50 ticks = 500 ms */
    }
}
/* USER CODE END 1 */
```

۳.۴ شرح اجزای کد

ساختار کنترلی Thread (TX_THREAD). هر Thread یک ساختار داده دارد که ThreadX وضعیت، اولویت و اشاره‌گر Stack آن را در آن نگه می‌دارد. این ساختار باید تا پایان برنامه زنده بماند؛ از این رو در سطح فایل و به‌صورت `static` تعریف شده است، نه داخل تابعی که پس از اجرا پایان می‌یابد.

Stack و Byte Pool. هر Thread متغیرهای محلی و نشانی بازگشت توابع خود را در Stack اختصاصی نگه می‌دارد. ThreadX خود Stack را نمی‌سازد؛ برنامه‌نویس باید حافظه آن را فراهم کند. در این پروژه حافظه از Byte Pool گرفته می‌شود که همان استخر ۱۶۳۸۴ بایتی تنظیم‌شده در CubeMX است. پارامتر `memory_ptr` اشاره‌گری به این استخر است و با تبدیل نوع به `TX_BYTE_POOL *` قابل استفاده می‌شود. اگر این تبدیل انجام نشود، متغیر `byte_pool` شناخته‌شده نخواهد بود و خطای `undeclared` رخ می‌دهد. گزینه `TX_NO_WAIT` یعنی در صورت نبودن حافظه کافی، تابع منتظر نمی‌ماند و بلافاصله خطا برمی‌گرداند؛ این رفتار در زمان راه‌اندازی مطلوب است.

اندازه Stack. مقدار ۲۰۴۸ بایت عمداً بزرگ انتخاب شده است، زیرا `printf` با قالب‌بندی اعداد مصرف Stack قابل‌توجهی دارد. کوچک بودن Stack از عوامل رایج رفتار غیرمنتظره یا Hard Fault است.

تابع tx_thread_create. پارامترهای این تابع، به ترتیب، در جدول زیر آمده است:

پارامتر	معنا
`&blink_thread`	ساختار کنترلی Thread که پیش‌تر تعریف شد
`"blink"`	نام Thread؛ فقط برای شناسایی در دیباگر کاربرد دارد
`blink_thread_entry`	تابعی که Thread اجرا می‌کند
`0`	مقدار ورودی دلخواهی که به تابع Thread ارسال می‌شود
`stack_ptr`، `BLINK_STACK_SIZE`	نشانی و اندازه حافظه Stack
`BLINK_PRIORITY` (نخست)	اولویت Thread
`BLINK_PRIORITY` (دوم)	آستانه preemption؛ تعیین می‌کند کدام Threadها می‌توانند این Thread را قطع کنند. فعلاً برابر اولویت قرار می‌گیرد که حالت معمول است
`TX_NO_TIME_SLICE`	بدون تقسیم زمانی؛ Thread تا زمانی که خود منتظر نشود یا Thread مهم‌تری آماده نشود، اجرا می‌گردد
`TX_AUTO_START`	Thread بلافاصله پس از ساخت آغاز به کار می‌کند

اولویت‌ها. در ThreadX عدد کوچک‌تر به معنای اولویت بالاتر است. اولویت صفر مهم‌ترین و اولویت ۳۱ کم‌اهمیت‌ترین سطح است (به دلیل مقدار `TX_MAX_PRIORITIES` برابر ۳۲). مقدار ۱۰ در این پروژه در میانه این بازه قرار دارد.

تابع ورودی Thread. این تابع «برنامه» Thread است و باید هرگز بازنگردد؛ از این رو دارای حلقه بی‌پایان است. بازگشت تابع به رفتار تعریف‌نشده می‌انجامد.

تابع tx_thread_sleep. این فراخوانی مهم‌ترین نکته کد است. با آن Thread به کرنل اعلام می‌کند که برای ۵۰ تیک به پردازنده نیاز ندارد، و کرنل پردازنده را به Threadهای دیگر می‌دهد. چون هر تیک برابر ۱۰ میلی‌ثانیه است (۱۰۰ تیک در ثانیه)، ۵۰ تیک معادل ۵۰۰ میلی‌ثانیه خواهد بود.

> **تفاوت بنیادین:** `HAL_Delay` پردازنده را مشغول نگه می‌دارد و در این مدت هیچ کار دیگری انجام نمی‌شود، در حالی که `tx_thread_sleep` Thread را موقتاً از زمان‌بند خارج می‌کند و پردازنده را برای Threadهای دیگر آزاد می‌سازد. همین ویژگی دلیل اصلی ارزش سیستم‌عامل بلادرنگ است: با یک پردازنده می‌توان چندین کار را به‌صورت هم‌زمان مدیریت کرد.

واحد زمان. تابع `tx_time_get` مقدار تیک ThreadX را برمی‌گرداند که واحد آن ۱۰ میلی‌ثانیه است، نه یک میلی‌ثانیه. به همین دلیل اعداد چاپ‌شده در این مرحله (۰، ۵۰، ۱۰۰ و ...) ده برابر کوچک‌تر از اعداد مرحله پیشین هستند که از `HAL_GetTick` می‌آمدند.

۳.۵ خروجی مورد انتظار

پس از ساخت و برنامه‌ریزی، LED قرمز هر نیم‌ثانیه تغییر وضعیت می‌دهد و خروجی ترمینال مشابه نمونه زیر خواهد بود. افزایش ۵۰ واحدی در هر خط نشان‌دهنده گذشت ۵۰۰ میلی‌ثانیه است.

خروجی ۲ — نمونه خروجی با ThreadX

```
tick 0
tick 50
tick 100
tick 150
tick 200
tick 250
```

۴. عیب‌یابی

جدول زیر رایج‌ترین مشکلات این مرحله و راه‌حل هر یک را فهرست می‌کند.

نشانه	علت محتمل	راه‌حل
خطای `byte_pool undeclared`	متغیر `byte_pool` در کد تولیدشده وجود ندارد	ساخت `byte_pool` از روی پارامتر `memory_ptr` در ابتدای بخش `App_ThreadX_MEM_POOL` (کد ۹)
خطای `errno` یا `STDERR_FILENO` تعریف‌نشده	سرفایل‌های `errno.h` و `sys/unistd.h` در این پروژه به‌درستی شناسایی نمی‌شوند	استفاده از نسخه ساده تابع `_write` (کد ۱) که به این سرفایل‌ها نیازی ندارد
خطای `multiple definition of _write`	فایل `syscalls.c` تعریف خود را دارد	حذف تابع `_write` و استفاده از روش جایگزین کد ۲
نمایش کاراکترهای نامفهوم	ناهماهنگی نرخ ارسال یا تنظیم نادرست کلاک UART۷	بررسی نرخ ۱۱۵۲۰۰ در ترمینال و بازبینی کلاک در CubeMX
نبودن هرگونه خروجی	جابه‌جایی سیم‌های TX و RX، نبودن GND مشترک، یا انتخاب نادرست پایه TX	بررسی سیم‌کشی و اطمینان از اینکه کدام یک از `PB3` و `PB4` در CubeMX به‌عنوان TX تنظیم شده است
توقف برنامه یا رفتار نادرست بلافاصله پس از راه‌اندازی	Timebase روی SysTick مانده است	تغییر `Timebase Source` به `TIM6` یا `TIM7`
رفتار عجیب یا Hard Fault پس از افزودن Thread	Stack یا Memory Pool کوچک است	افزایش اندازه Stack (حداقل ۲۰۴۸ بایت) و Memory Pool (۱۶۳۸۴ بایت)
LED چشمک نمی‌زند	پایه نادرست یا نبودن تنظیم خروجی	بررسی نام پایه و حالت GPIO؛ توجه به اینکه LED ممکن است active-low باشد

۵. معیار پایان مرحله و گام بعدی

مرحله صفر زمانی کامل محسوب می‌شود که شرایط زیر همگی برقرار باشند:

- پروژه ThreadX بدون خطا ساخته و روی برد برنامه‌ریزی شود.

- LED قرمز هر ۵۰۰ میلی‌ثانیه توسط یک Thread تغییر وضعیت دهد.
- پیام تیک سیستم هر بار روی ترمینال UART۷ چاپ شود.
- کاربر بتواند نقش هر بخش از کد (ساختار Thread، Stack، `tx_thread_create`، `tx_thread_sleep`) را توضیح دهد.

برای تثبیت یادگیری، تمرین پایانی این مرحله ساخت **Thread دوم** است که LED سبز (`PC9`) را هر ۱۰۰۰ میلی‌ثانیه تغییر وضعیت دهد، مستقل از Thread نخست. برای انجام آن لازم است:

- یک ساختار `TX_THREAD` جدید تعریف شود.
- Stack دیگری با `tx_byte_allocate` از Byte Pool گرفته شود (دو Stack با اندازه ۲۰۴۸ بایت در مجموع ۴۰۹۶ بایت مصرف می‌کنند و در استخر ۱۶۳۸۴ بایتی جای می‌گیرند).
- فراخوانی دوم `tx_thread_create` با نام و تابع ورودی جدید انجام شود.
- تابع ورودی جدیدی نوشته شود که پس از هر تغییر وضعیت LED، به اندازه ۱۰۰ تیک در `tx_thread_sleep` بماند.

پس از موفقیت این تمرین، گام بعدی مرحله یکم برنامه آموزشی است: هماهنگی میان Threadها با **Mutex، Queue، Semaphore و Event Flags** و ارتباط میان وقفه‌ها (ISR) و Threadها.